Aluno(a): Allan Djones Costa Hladki
Orientador(a): Carlos Henrique Rodrigues Sarro

Curso: MBA em Engenharia de Software

Desenvolvimento de biblioteca Java para converter textos posicionais em POJOs: Prova de conceito com API Rest

Introdução

Sistemas de *software*, assim como sistemas físicos, tendem a se desestabilizar e ficarem caóticos ao longo do tempo, por fatores como a degradação do equipamento utilizado e aumento de complexidade em manutenções (ou a falta desta), fazendo com que cresçam os custos e tempos de correções e aperfeiçoamentos, também podendo gerar riscos de inconsistência e mau funcionamento (Sakthivel, 2023).

Também é natural que tecnologias antigas demandem maiores custos que implementações novas, devido a otimizações e novas abordagens na engenharia de *software*, fazendo com que, segundo Nitin (2024), surja a necessidade de modernização.

Apesar dos crescentes esforços na atualização digital, ainda é encontrado uma grande quantidade de código sendo implementado e mantido em tecnologias legadas. Segundo pesquisa da Micro Focus (2022), em 2022, ainda existiam mais de 800 bilhões de linhas de código COBOL em uso, com 92% das empresas considerando essas aplicações estratégicas. Além disso, 64% das organizações pesquisadas não desejam substituir seus sistemas, e sim, modernizá-los, com 43% pretendendo integrar o *mainframe* à nuvem.

Atualmente, está em evidência a transcrição de código com a utilização de ferramentas de Inteligência Artificial (IA), como elencado no trabalho de Nitin (2024), todavia, não é de hoje que existem iniciativas para o desenvolvimento de ferramentas para migração e integração entre ambientes, como as soluções apresentadas por Meziane e Aliwa, (2003) e Wirth (2010).

Segundo Coughlan (2014), em circunstâncias restritas, uma total reimplementação de um sistema em uma outra linguagem é muitas vezes uma estratégia viável de modernização, principalmente quando a documentação do programa original está disponível, no entanto, muitas vezes não é este o caso, fazendo com que a integração entre legado e novas aplicações seja uma alternativa para retirar partes da solução de tecnologias datadas.

Nesse cenário, uma possibilidade para revitalização de sistemas onde a migração apresenta muitos riscos, e é necessária a integração com outros ambientes, pode ser a utilização de interfaces assíncronas de comunicação, como filas, tópicos, bancos de dados ou arquivos, ou estratégias proprietárias, como a arquitetura de rede em nuvem híbrida, da empresa IBM, onde uma aplicação fora do ambiente z/OS transita dados ou arquivos no centralizado, ou um programa *mainframe* acessa repositórios externos (Parziale et al., 2023).

Na utilização de interfaces de comunicação através de filas, tópicos ou arquivos, é necessário que ambas as aplicações se comuniquem através do mesmo protocolo e formato, fazendo com que a mensagem seja inteligível em ambas as pontas.

O COBOL, tem sua origem em 1959, e apesar de em suas últimas atualizações contar com conceitos como a Orientação à Objetos (OO), tem sua maior utilização em variações mais antigas, no modelo estruturado, rodando em *mainframes* (Murach, 2004).

Segundo Coughlan (2014), nessa linguagem, existem apenas dados numéricos, alfanuméricos (texto/*string*), e alfabéticos. Quando esses elementos são declarados como variáveis, internamente são definidos os espaços de armazenamento, onde, no geral, cada dígito ou caractere utiliza um byte (Murach, 2004).

Logo, é limitada a liberdade que o desenvolvedor tem de processar tipos mais sofisticados de informações de forma nativa, assim como pode ser complexa a transação de outros tipos de formatos que não o alfanumérico em processos *input/output* (I/O).

Uma forma de contornar essas limitações, seria a aplicação que se comunica com o *mainframe* tratar os dados no formato recebido. Para isso, pode ser utilizado o processo de serialização, onde, segundo Schildt (2022), a referência original é transformada em uma sequência de bytes, e então, pode ser desserializada na mesma ou em outra estrutura, mantendo as informações originais.

É comum em programas COBOL, de acordo com Coughlan (2014), utilização de itens de grupo, onde o tamanho do elemento final é a soma do tamanho de seus subordinados, e o seu tipo é alfanumérico. Como cada subordinado ocupa um tamanho pré-definido, se não forem utilizados binários computacionais nas entradas e saídas, o resultado do processamento pode ser interpretado como um texto posicional, onde um número específico de caracteres representa um elemento ou subordinado.

Então, aplicando a serialização e desserialização de textos posicionais em elementos nativos na aplicação mais flexível, é possível a comunicação assíncrona entre esses ambientes.

Schildt (2022) aponta que o Java é uma linguagem desenvolvida no início dos anos 90, todavia, segundo a pesquisa anual *Octoverse* do GitHub (2024), em 2024 ela foi a quarta mais utilizada em contribuições de código, sendo a terceira com mais repositórios criados.

Nesse contexto, sendo essa uma linguagem atual e amplamente utilizada, é proposto o desenvolvimento de uma biblioteca *open-source* de serialização e desserialização de textos posicionais em *Plain Old Java Objects* (POJOs), ou simplesmente objetos Java, para que seja possível o processamento de informações oriundas do *mainframe* ou qualquer outra fonte que utilize esse formato, na nuvem, e vice-versa, validando a implementação através de uma prova de conceito utilizando uma API Rest.

Objetivo

O Objetivo do trabalho é desenvolver uma biblioteca Java para a conversão de textos posicionais em POJOs, e conversão de POJOs em textos posicionais, e então, através de uma API Rest desenvolvida na mesma linguagem, realizar prova de conceito, testando diferentes casos para validações do funcionamento dessas transformações.

Metodologia

A pesquisa será descritiva, sendo um estudo de caso, expondo o funcionamento da biblioteca e detalhando a conversão, sendo qualitativa-quantitativa, devido a explicação dos métodos, assim como o resultado das chamadas HTTP aos *endpoints*.

Para a criação da biblioteca será utilizada a versão 21 do Java, última com suporte à longo prazo (*Long-Term Support* – LTS) e o gerenciador de pacotes e dependências Apache Maven 3.9.9.

Será almejado a criação de uma anotação através de uma interface Java, onde atributos anotados sejam identificados e validados em tempo de compilação, de forma a garantir que a transformação seja possível apenas em tipos esperados.

Dessa forma, será permitido a serialização de atributos das classes java.lang.Integer, java.lang.Long, java.math.BigInteger, java.math.BigDecimal, java.lang.String, java.time.LocalDate e java.time.LocalDateTime.

Especificamente, para o tipo BigDecimal será exigido a especificação da quantidade de casas decimais, assim como para os atributos LocalDate e LocalDateTime será opcional a informação do formato de data. Não será permitido a serialização ou desserialização de atributos *Long,* visto o COBOL ser naturalmente uma linguagem de programação com casa decimal de ponto fixo, podendo sujeitar a aplicação a erros de arredondamento.

A transformação se dará em tempo de execução, através de métodos estáticos que recebem e retornam tipos genéricos, mantendo a versatilidade.

Para a criação dos textos posicionais, será utilizado a solução GnuCobol, versão 3.2, possibilitando a execução de código fora do ambiente *mainframe*.

Ressalta-se que nesta linguagem, os campos numéricos, declarados como *Picture 9,* são preenchidos com zeros à esquerda até seu tamanho, enquanto os campos alfanuméricos, declarados como *Picture X* são preenchidos com espaços à direita até o número de caracteres esperado. Também será criado uma configuração de ordenação, visto a ordem dos atributos alterar o resultado.

A Figura 1 apresenta um exemplo de objeto Java com alguns atributos, e sua correspondência no COBOL, assim como o texto posicional que seria resultado da serialização, utilizando os valores comentados como exemplo.

Java

```java
class Carro {

    //Exemplo: AGG-1133
    //Tamanho: 8
    String placa;

    //Exemplo: 2019
    //Tamanho: 4
    Integer anoFabricacao;

    //Exemplo: HB20
    //Tamanho: 30
    String modelo;

    //Exemplo: 65400
    //Tamanho: 9
    Long quilometragem;
}
```

COBOL

```
01 CARRO.
   03 PLACA     PIC X(008) USAGE DISPLAY.
   03 ANO       PIC 9(004) USAGE DISPLAY.
   03 MODELO    PIC X(030) USASE DISPLAY.
   03 KM        PIC 9(009) USAGE DISPLAY.
```

Texto posicional: "AGG-11332019HB20 000065400"

Figura 1 - Representação de um objeto em texto posicional
Fonte: Dados originais da pesquisa

Para a prova de conceito será desenvolvida uma API Rest utilizando Java 21, e o *framework* Quarkus 3.15 (LTS) com a biblioteca de serialização Jackson para o formato JSON, disponibilizando 3 *endpoints* com os seguintes métodos HTTP:

- **[POST] /serialize**: Recebe o conteúdo em texto posicional (*plain-text)* e o converte em um POJO, imprimindo os atributos do objeto através da sobreposição do método *Object.toString()* e devolvendo um JSON.
- **[POST] /deserialize**: Recebe o conteúdo em JSON, convertendo-o em um objeto Java, desserializando-o em um texto posicional e retornando-o.
- **[GET] /file:** Busca um arquivo de texto em um diretório pré-definido, imprime e converte o seu conteúdo em um POJO, retornando o resultado como um JSON.

As classes que representam os POJOs serão definidas posteriormente, buscando contemplar todos os tipos de atributos passíveis de conversão, assim demonstrando diferentes configurações e ordenações.

Resultados Esperados

É esperado que com essa biblioteca seja possível a serialização e desserialização dinâmica e versátil de objetos em textos posicionais, assim, demonstrar-se-á através da prova de conceito sua aplicabilidade.

Deseja-se que sejam implementados os *endpoints* supracitados na API, e ao final, sejam esmiuçadas suas chamadas, com *logs* das etapas intermediárias e relatório descritivo e qualitativo das respostas, comparando-as com textos criados no COBOL como resultados esperados.

Não será escopo do trabalho aferir desempenho ou comparar arquiteturas, assim como não será implementada comunicação direta através de interface entre a API e o programa COBOL.

Cronograma de Atividades

Para este trabalho, é apresentado o cronograma de atividades do Quadro 1.

Quadro 1. Cronograma de Atividades

Atividades planejadas	**Mês**							
	mar/25	abr/25	mai/25	jun/25	jul/25	ago/25	set/25	out/25
Projeto de Pesquisa	X							
Revisão Bibliográfica	X	X	X	X	X	X		
Desenvolvimento da Biblioteca		X	X	X	X			
Prova de Conceito			X	X	X	X		
Resultados Preliminares			X	X				
Resultados e Discussões					X	X		
Entrega do TCC							X	
Entrega da Apresentação								X

Referências

Coughlan, M. 2014. Beginning COBOL for Programmers. 1ed. Apress, Berkley, CA, EUA. ISBN: 978-1-4302-6253-4.

GitHub. 2024. Octoverse: AI leads Python to top language as the number of global developers surges. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-2024/>. Acesso em: 16 mar. 2025.

Meziane, F.; Aliwa, J. 2003. From COBOL to SQL through program transformation and XML. In: Third International Conference on Information Technology in Asia, 2003, Kuching, Sarawak, Malásia. Anais p. 1-8. Disponível em: <https://salford-repository.worktribe.com/output/1468689/from-cobol-to-sql-through-program-transformation-and-xml>. Acesso em: 9 mar. 2025.

Micro Focus. 2022. Modernize on Strength: A Global COBOL Survey. Disponível em: <https://www.microfocus.com/media/infographic/modernize-on-strength-a-global-cobol-survey-infographic.pdf>. Acesso em: 9 mar. 2025.

Murach, M.; Prince, A.; Menendez, R. 2004. Murach's Mainframe COBOL. 1ed. Mike Murach & Associates, Incorporated, Fresno, California, EUA, 2004. ISBN-10. 1890774243.

Nitin, V. 2024. Using AI to automate the modernization of legacy software applications In: 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, Nova Iorque, NY, EUA. Anais p. 2514–2517. DOI: 10.1145/3691620.3695610. Disponível em: <https://ieeexplore.ieee.org/document/10764808>. Acesso em: 16 mar. 2025.

Parziale, L.; Adesanya, Y.; Souza, E.; Haumer, P.; Irmes, S.; Patil, A.; Li, L. K.; Li, L.; Miranda, F.; Varoni, S. 2023. Mainframe Application Modernization Patterns for Hybrid Cloud. 1ed. IBM Redbooks, EUA, 2023. Disponível em: <https://www.redbooks.ibm.com/abstracts/sg248532.html>. Acesso em: 16 mar. 2025.

Sakthivel, R. 2023. A critical review of cost-benefit analysis for business software investments. American Journal of Business 38(4): p. 229-247, 2023. DOI: 10.1108/ajb-09-2022-0145. Disponível em: <https://www.emerald.com/insight/content/doi/10.1108/ajb-09-2022-0145/full/html>. Acesso em: 16 mar. 2025.

Schildt, H. 2022. The Complete Reference Java. 12ed. McGraw Hill, EUA, 2022. ISBN: 978-1-26-046342-2.

Wirth, M. 2010. Migrating from COBOL to Java. In: IEEE International Conference on Software Maintenance, 2010, Timisoara, Romênia. Anais p. 1-7. Disponível em: <https://ieeexplore.ieee.org/document/5609583>. Acesso em: 9 mar. 2025.